2026

OVER ZHOUSANDS

DESARROLLO DE VIDEOJUEGOS Y ENTORNOS 3D

XINHAO XIA XIA

UNIVERSIDAD DE DISEÑO,
INNOVACIÓN Y TECNOLOGÍA

INDICE

Enlace del GitHub-> https://github.com/Sickgecko/Over_Zhousands 3
Descripción Técnica 3
Introducción y Rol Individual 3
Arquitectura Conceptual y Documentación de Diseño 3
Diseño de Mecánicas y Sistemas 4
Mecánica Central del Control Zombi 4
Gestión de Recursos y Construcción 5
Sistema de Enfrentamiento con Bases Militares 5
Árbol de Habilidades 5
Diseño del Flujo de Juego, Tutorial y Level 1 5
Flujo de Aprendizaje y Tutorial 5
Diseño del Nivel 1 5
Arte Conceptual y Dirección Visual 6
Continuación del Desarrollo 7
Documentación y Organización 7
Diseño de Cámara 7
Modelado 3D 8
Nivel y Escenarios 8
Nivel 1 8
Nivel 2 9
Nivel 3 9
Nivel 4 9
Ajustes Técnicos 10
QA y Testing 11
Diario de desarrollo y conclusiones 12
Anexos 14

Enlace del GitHub-> https://github.com/Sickgecko/Over_Zhousands

Descripción Técnica

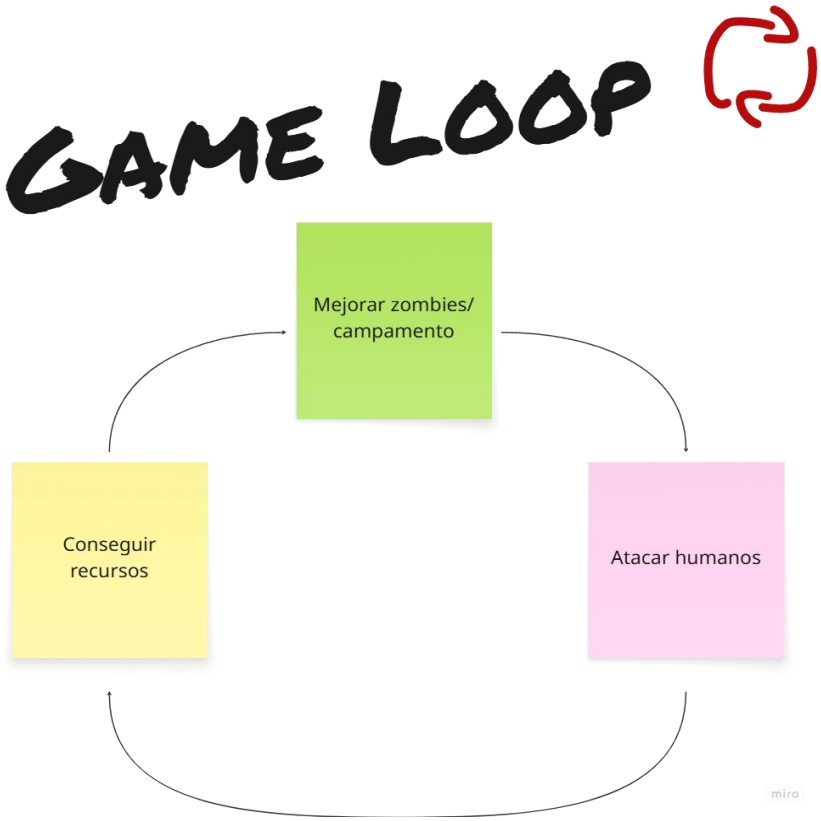
DISEÑADOR DE JUEGO

Introducción y Rol Individual

El rol desempeñado en el proyecto fue el de **Diseñador del Juego** dentro del equipo de desarrollo de *Over Zhousands*, un título de gestión de recursos y ejército cuyo eje diferenciador reside en que el jugador controla **hordas de zombis** en lugar de humanos, lideradas por un único zombi consciente capaz de organizar, construir y expandirse para conquistar el mundo y enfrentar a bases militares que buscan una cura.

Las responsabilidades individuales abarcaron el diseño sistémico, conceptual, estructural y visual del juego, incluyendo documentación formal, definición del flujo lúdico, diseño de niveles, mecánicas, inputs, interfaces, así como parte de la dirección artística conceptual.

Arquitectura Conceptual y Documentación de Diseño



Se elaboró un conjunto amplio de documentos de diseño almacenados en **OneDrive**, donde el equipo organizó todos los materiales de diseño, arte, producción y programación.
Los documentos desarrollados individualmente fueron:

- **Diseño del Árbol de Habilidades**: estructura progresiva, ramificación, dependencias y costes.

Diseño Arbol Habilidades.docx

- **Diseño del Nivel 1**: objetivos, ritmo, distribución espacial y curva inicial de dificultad.

Diseño de Nivel 1.docx

- **Diseño del Tutorial y Flujo de Aprendizaje**: estructura pedagógica y secuencia óptima de introducción; este documento se realizó tardíamente al no contar aún con todos los niveles documentados.

Diseño de Tutorial y Flujo de Aprendizaje.docx

- **Diseño de Inputs y Sistema de Control:** tanto para la base zombi y la base militar, se hicieron unas tablas con todos los controles del juego.

Inputs y Sistema de Control.docx

- **Diseño de Mecánicas del Juego**: descripción detallada de todas las mecánicas primarias, secundarias y de apoyo.

Mecanicas.docx

- **Productos Transmedia de la IP**: proyección futura (cómic, cartas, productos narrativos, etc.).

Productos de la IP.docx

- **Concept Art Book del Proyecto**: preparación y ordenación estructural del libro de arte oficial.

VER1.0_Concept Art Book Over Zhousands.docx

Estos documentos establecieron la base formal del producto, sirviendo como referencia para programación, arte y modelado.

Diseño de Mecánicas y Sistemas

La labor de diseño incluyó la definición del núcleo sistémico del juego, destacando:

Mecánica Central del Control Zombi

- Control indirecto de unidades mediante un "zombi consciente" que lidera la horda.
- Mecanismos de obediencia, propagación y control masivo.
- Integración con la base zombi y su expansión.

Gestión de Recursos y Construcción

- Diseño de edificios de la base zombi y de la base militar (concept art y organización funcional).
- Definición de costes, tiempos de construcción y progresión tecnológica.

Sistema de Enfrentamiento con Bases Militares

- Estructura de oleadas, comportamiento enemigo, puntos débiles y objetivos prioritarios.
- Diseño de escalado de dificultad por zonas.

Árbol de Habilidades

- Diagrama funcional creado en Miro.
- Equilibrio entre mejoras pasivas, activas y progresiones estratégicas.

Diseño del Flujo de Juego, Tutorial y Level 1

Miro

Flujo de Aprendizaje y Tutorial

- Se diseñó la secuencia óptima para introducir mecánicas.
- Se definieron hitos de aprendizaje, refuerzos y pruebas de competencia.
- El documento fue realizado al final del proceso, aunque debería haberse generado tras completar todos los diseños de niveles, hecho que se reconoce como un error de planificación de diseño.

Diseño del Nivel 1

- Se definieron objetivos principales, distribución espacial y ritmo.
- Se generó un **Whitebox en Unity 6** (pendiente de finalización), con pruebas de flujo y ajustes de variables para optimizar la experiencia.

- Se estableció la integración de mecánicas ya completadas (80–90%) para validar la jugabilidad.

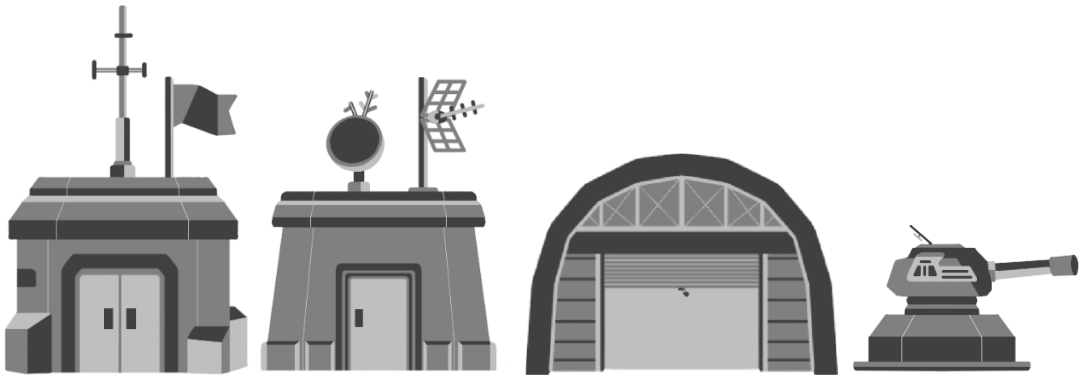
Arte Conceptual y Dirección Visual

Además de las tareas de diseño, se contribuyó activamente al apartado artístico:

- Concept art de **edificios de la base zombi**, aquí algunos ejemplos:



- Concept art de **edificios de la base militar enemiga,** aquí algunos ejemplos:



- Participación en el diseño visual del mundo, personajes y colorimetría mediante Miro.
- Preparación de la estructura y organización del **Concept Art Book** del proyecto.
- Se asumió, además, parte del **modelado 3D** de edificios que iré haciendo.

Continuación del Desarrollo

Tras la entrega inicial del proyecto, se han seguido realizando diferentes tareas centradas en la implementación del diseño dentro del motor, el testeo del juego y la mejora de sistemas ya planteados anteriormente.

Documentación y Organización

Se ha ido ampliando la documentación del proyecto con cosas más enfocadas al funcionamiento interno del juego:

- Se ha realizado un Excel con enemigos y aliados, donde se recogen sus estadísticas principales.

Aliados y Enemigos.xlsx

- Se ha hecho un documento con los valores de todas las unidades después de testear en el editor.
- Se ha organizado un Notion general con todas las unidades del juego (zombis, humanos y torretas), incluyendo sus estadísticas.

Notion

- Se ha anotado QA con las observaciones del juego según las pruebas realizadas.
- Se ha terminado el documento de la cámara y como seria sin el personaje.

Diseño Cámara Campamento.docx



Battlefield:

Player Battlefield (Script)

Script	PlayerBattlefield
Speed_move	9
Camera_transform	Main Camera (Transform)
Retroceso Movimiento	
Duracion_reversa	0.4
Speed_retroceso	2

- Velocidad jugador: 9
- Duración reversa: 0.4
- Velocidad retroceso: 2

ZOOM (SCROLL)

El jugador podrá ajustar la distancia de la cámara mediante la rueda del ratón:

- Zoom In → Girando la rueda hacia arriba
- Zoom Out → Girando la rueda hacia abajo

La cámara tendrá:

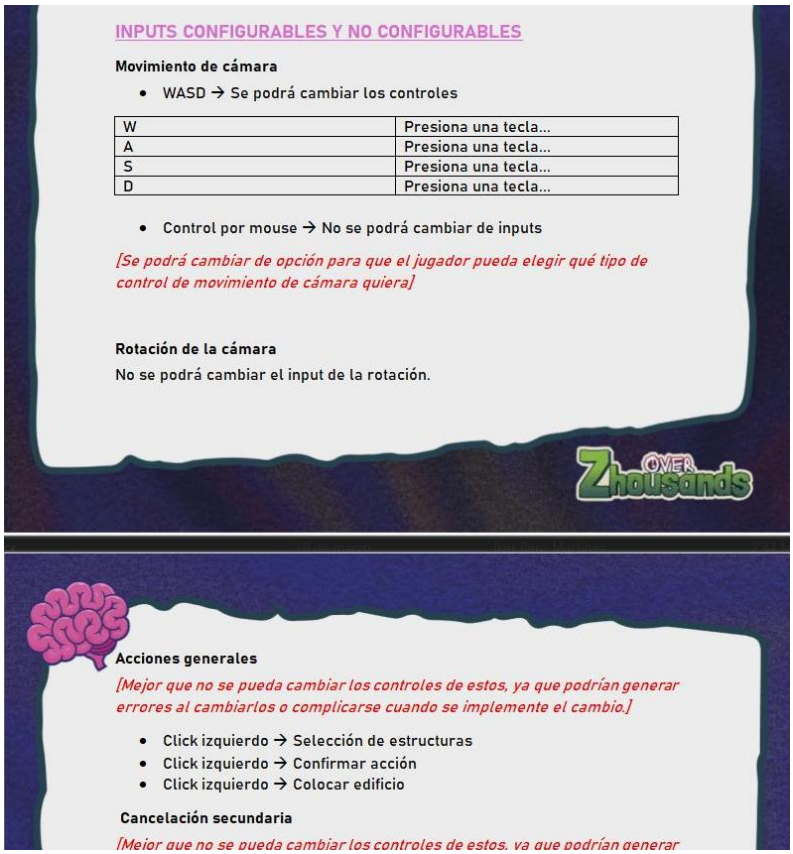
Camera_Script (Script)

Script	Camera_Script
Target	Player (Transform)
Angle	727.3904
Sensitivity	21.5
Distance	30
Min_zoom	5
Max_zoom	15
Zoom_sensitivity	0.85

- Sensiblilidad: 21.5
- Distancia: 30
- Zoom mín: 5
- Zoom máx: 15
- Sensibilidad zoom: 0.85

Esto evita perder visibilidad del Campamento o comprometer la precisión al colocar estructuras.

- Se ha realizado un documento de cambios de controles y opciones, guardado en OneDrive.



INPUTS CONFIGURABLES Y NO CONFIGURABLES

Movimiento de cámara

- WASD → Se podrá cambiar los controles

W	Presiona una tecla...
A	Presiona una tecla...
S	Presiona una tecla...
D	Presiona una tecla...

- Control por mouse → No se podrá cambiar de inputs

[Se podrá cambiar de opción para que el jugador pueda elegir qué tipo de control de movimiento de cámara quiera]

Rotación de la cámara

No se podrá cambiar el input de la rotación.

Acciones generales

[Mejor que no se pueda cambiar los controles de estos, ya que podrían generar errores al cambiarlos o complicarse cuando se implemente el cambio.]

- Click izquierdo → Selección de estructuras
- Click izquierdo → Confirmar acción
- Click izquierdo → Colocar edificio

Cancelación secundaria

[Mejor que no se pueda cambiar los controles de estos, ya que podrían generar

Diseño de Cámara

Se ha trabajado en el diseño de la cámara del juego, orientada a un sistema sin personaje:

- Se ha definido cómo funciona el control y movimiento de la cámara.
- Se ha tenido en cuenta que el juego es de gestión y no de personaje directo.
- El documento se ha revisado con el equipo de programación para su implementación.

Modelado 3D

Se ha empezado a trabajar en modelado 3D como apoyo al apartado artístico:

- Se ha hecho el modelo 3D del edificio "taller" y se ha exportado en FBX.
- El modelo no está texturizado por falta de tiempo.
- No se han hecho más modelos por falta de experiencia en esta parte.



Esta parte ha sido complicada ya que no tenía experiencia previa en modelado 3D.

Nivel y Escenarios

Se ha trabajado en la implementación de niveles dentro del juego, llevando a la práctica lo planteado en los documentos de diseño:

- Se han creado cuatro niveles pequeños y diferentes entre sí, cada uno para probar situaciones distintas.

Nivel 1



Nivel 2



Nivel 3



Nivel 4



- En estos niveles se han colocado edificios, torretas y zonas de paso para que el jugador tenga que gestionar el avance de las unidades.
- Cada nivel tiene una distribución distinta, cambiando la colocación de estructuras y enemigos para variar la forma de jugar.
- Se ha modificado el terreno de la escena de combate para mejorar la navegación.
- Se ha trabajado con el **NavMesh** para que las unidades se muevan correctamente por el escenario, así que los zombis no se queden pillado cuando va a atacar con las estructuras.

Además, se han añadido estructuras destruibles dentro de los niveles:

- Estas estructuras forman parte del objetivo principal del jugador en los ataques.
- Se han colocado en zonas estratégicas para añadir dificultad y obligar a planificar el ataque.

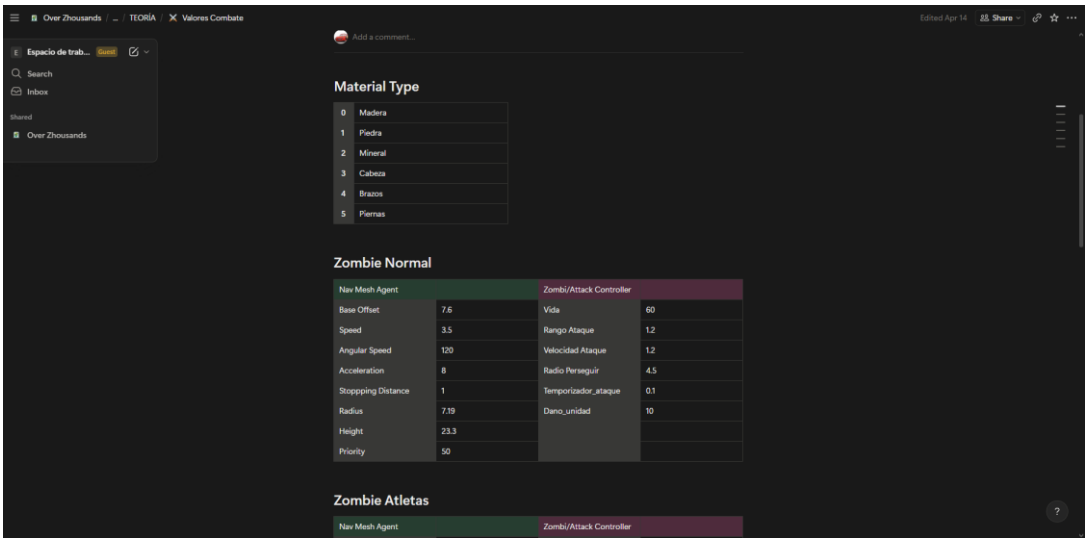
Ajustes Técnicos

Se han realizado diferentes ajustes en sistemas del juego:

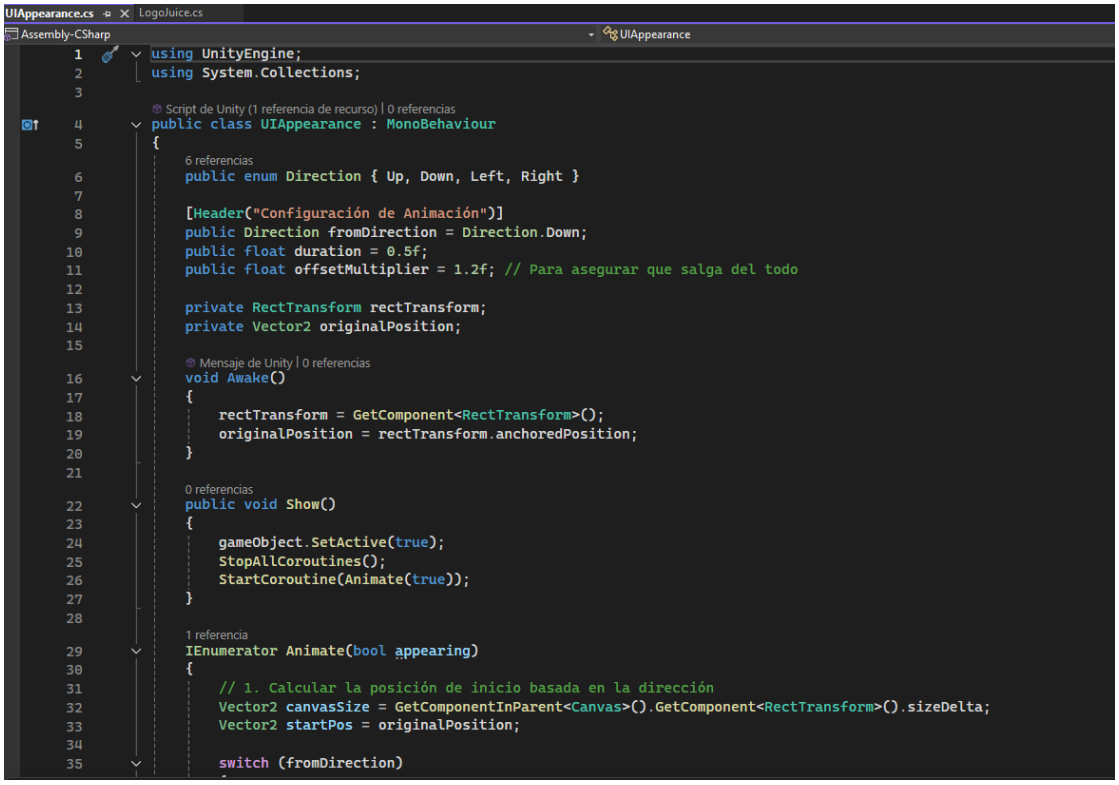
Notion

Descripción Tooltip y Valores Iniciales

- Se han corregido las colisiones de zombis y humanos para evitar que se queden atascados.
- Se han ajustado los valores de las unidades, tanto los diferentes tipos de zombis y humanos, y la torreta.



- Se ha añadido **Script** para animar elemento de la UI, como el menú de construcción.



```
using UnityEngine;
using System.Collections;

Script de Unity (1 referencia de recurso) | 0 referencias
public class UIAppearance : MonoBehaviour
{
    6 referencias
    public enum Direction { Up, Down, Left, Right }

    [Header("Configuración de Animación")]
    public Direction fromDirection = Direction.Down;
    public float duration = 0.5f;
    public float offsetMultiplier = 1.2f; // Para asegurar que salga del todo

    private RectTransform rectTransform;
    private Vector2 originalPosition;

    Mensaje de Unity | 0 referencias
    void Awake()
    {
        rectTransform = GetComponent<RectTransform>();
        originalPosition = rectTransform.anchoredPosition;
    }

    0 referencias
    public void Show()
    {
        gameObject.SetActive(true);
        StopAllCoroutines();
        StartCoroutine(Animate(true));
    }

    1 referencia
    IEnumerator Animate(bool appearing)
    {
        // 1. Calcular la posición de inicio basada en la dirección
        Vector2 canvasSize = GetComponentInParent<Canvas>().GetComponent<RectTransform>().sizeDelta;
        Vector2 startPos = originalPosition;

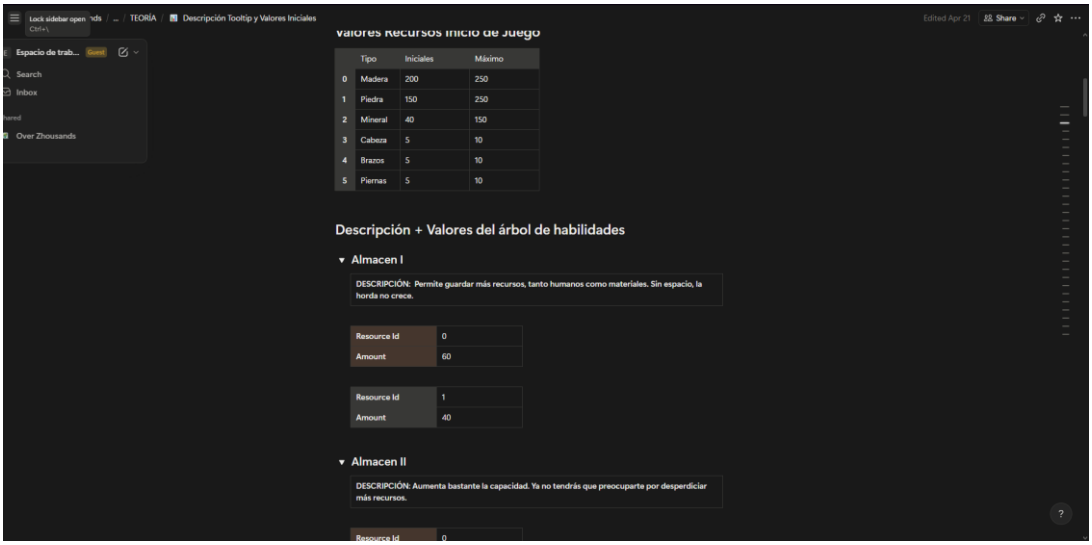
        switch (fromDirection)
```

Lo que hace es mover el objeto desde fuera de la pantalla hasta su posición original usando una animación suave.

Primero guarda la posición original del elemento. Cuando se activa, calcula desde qué dirección va a entrar (arriba, abajo, izquierda o derecha) y coloca el objeto fuera de la pantalla.

Luego, mediante una corrutina, lo mueve poco a poco hacia su posición usando un *Lerp* con suavizado, para que la animación no sea brusca.

- Se ha creado una animación (gif) para el paso de día.
- Se ha añadido una animación tipo respiración y una ligera rotación de lado a lado, al logo en el menú principal.
- Se hizo un documento de las descripciones y valores iniciales de cada estructura del árbol de habilidades.



Uno de los problemas más importantes fue que la mecánica de destrucción de edificios no funcionaba correctamente. Al principio se dejó de lado porque no se conseguía hacer funcionar bien. Más adelante se decidió solucionarlo:

- Se sustituyeron los assets por otros nuevos.
- Se colocaron los elementos para que el edificio pueda ser destruido y que se vieran las barras de vida de cada una de las estructuras y poner las cantidades de elementos de sueltan por ser destruidos.
- Se ajustó la escala correctamente.
- Se volvieron a colocar los elementos en la escena.

A partir de ese momento, la destrucción de edificios empezó a funcionar correctamente, permitiendo aplicar la mecánica como estaba planteada.

QA y Testing

Se ha realizado testing del juego para detectar errores:

- Se ha hecho QA del nivel, identificando problemas.
- Se ha recogido todo en un documento.

- Se han hecho cambios en el UI de la escena de campamento a partir de las pruebas.

Diario de desarrollo y conclusiones

Mirando todo el proceso con un poco de distancia, me doy cuenta de que este proyecto ha sido más un viaje personal que una simple práctica. Ha habido momentos de avance, otros de atasco total, y muchas dudas en el camino. Uno de los principales obstáculos ha sido enfrentarme a áreas que no manejo del todo o en las que, directamente, no tengo casi base. El diseño de niveles, por ejemplo, me gusta, pero la verdad es que no tengo tanto conocimiento como me gustaría, y en clase apenas lo hemos tocado todavía. Sé que lo daremos más adelante, pero mientras tanto me ha tocado improvisar más de lo que me habría gustado.

A eso se suma que el proyecto tiene mecánicas y elementos de géneros que no suelo consumir: la gestión de recursos, estrategia, manejo de ejércitos… No es lo mío. Yo estoy más conectado con los juegos de terror, psicológicos, de misterio, de puzles… ese tipo de experiencias más íntimas, tensas o inquietantes. Y claro, cuando te toca trabajar con algo que no forma parte de tu "zona natural", pues se nota el desequilibrio. Me faltan referencias, me faltan ejemplos, y a veces incluso la motivación cuesta un poco más.

Aun así, creo que esto también ha tenido su parte positiva. Me ha obligado a salir de lo cómodo y a pensar de otra manera. He tenido que buscar soluciones, probar cosas, aunque no estuviera seguro, y aprender a base de error. En el proceso me he dado cuenta de que muchas veces no tienes que saber hacerlo todo desde el principio, pero sí tienes que ser capaz de ir resolviéndolo poco a poco.

También he aprendido a organizarme mejor dentro de lo que cabe. Al principio iba un poco más perdido, pero con el tiempo he ido entendiendo mejor qué tenía que hacer como diseñador, qué decisiones eran importantes y cuáles no tanto, y sobre todo qué cosas necesitaban más tiempo del que pensaba.

Si hago un balance, diría que el proyecto me ha mostrado con bastante claridad quién soy como desarrollador ahora mismo. Y aunque eso incluye carencias, también me da una dirección bastante concreta para mejorar. Ya no es un "sé que me falta algo", sino un "sé exactamente qué me falta".

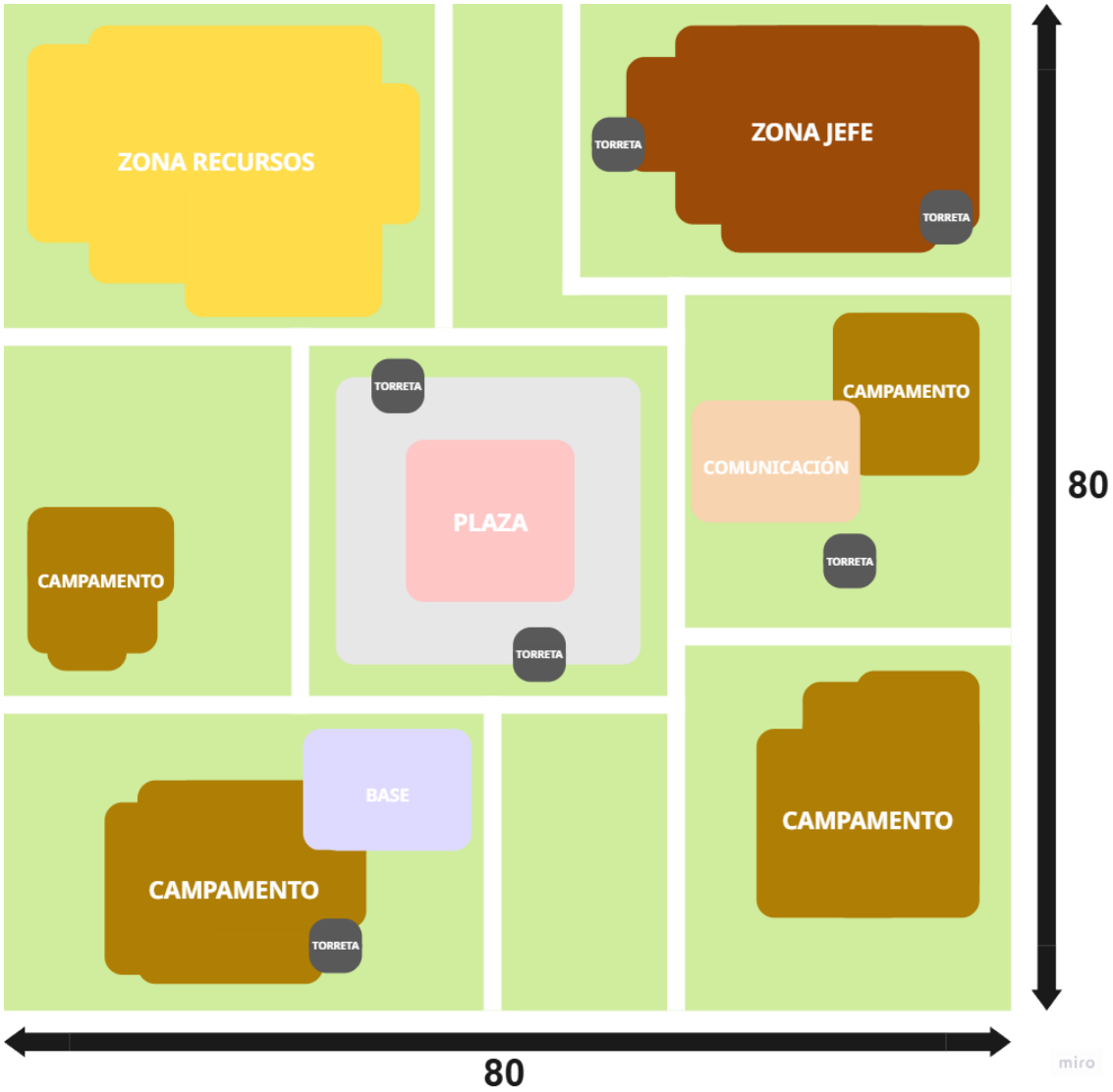
Conclusiones

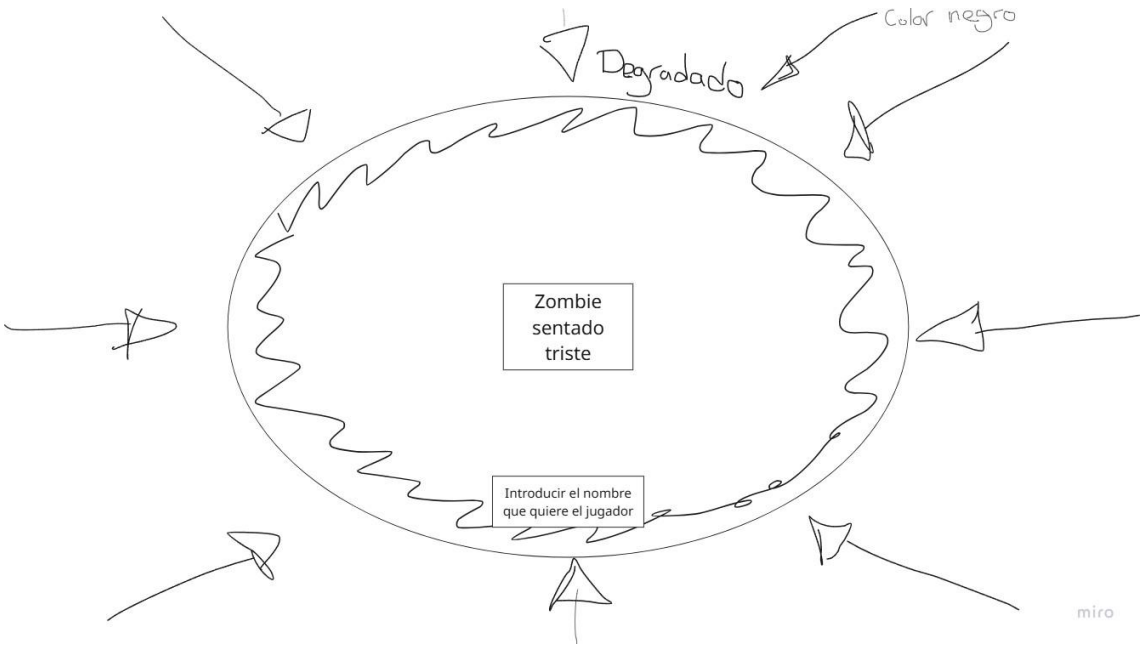
— Tengo interés real en el diseño de niveles, pero necesito más teoría y práctica para poder aplicarlo bien.
— Los juegos psicológicos, de terror y puzles son mi terreno natural, pero no puedo limitarme solo a eso si quiero crecer.
— Trabajar en géneros que no me gustan tanto también tiene valor, porque te obliga a entender otros sistemas de diseño.
— Avanzar sin una base sólida es posible, pero hace que todo el proceso sea más lento y difícil.

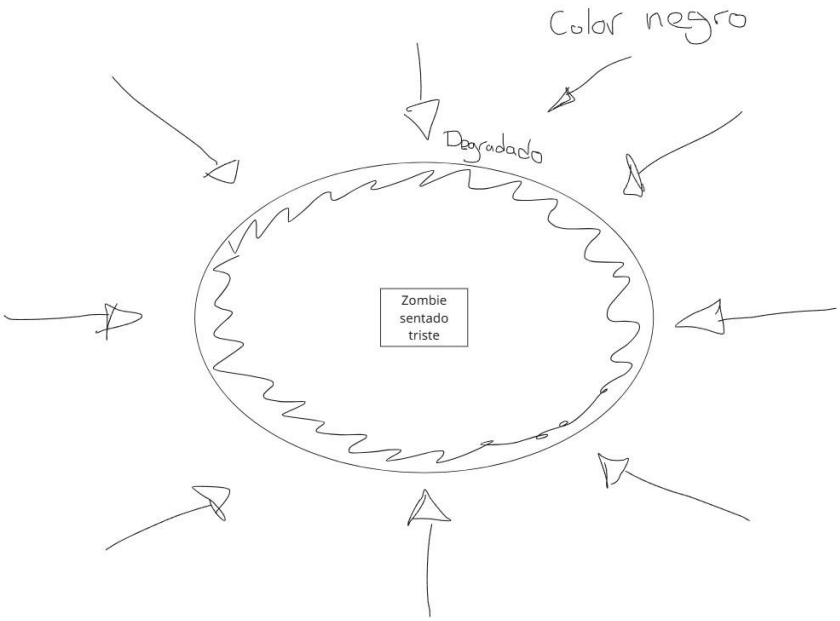
Próximos pasos

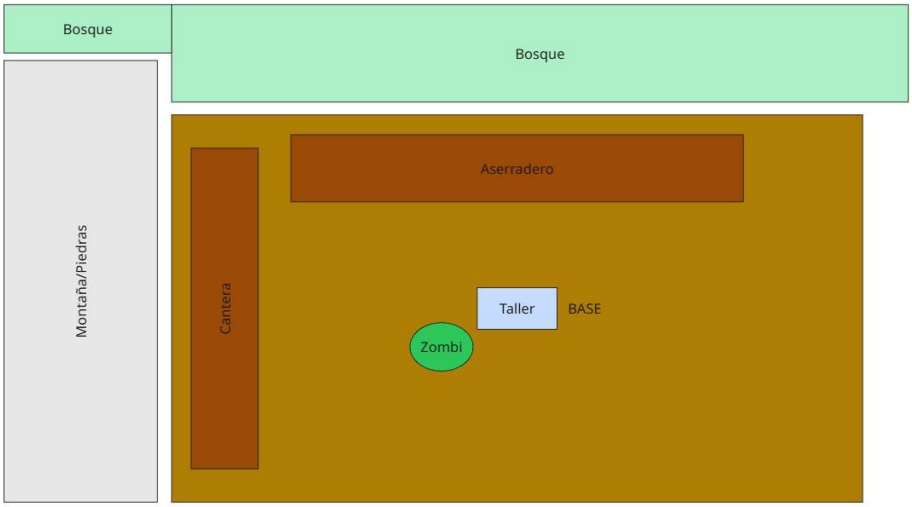
— Aprovechar cuando demos diseño de niveles en clase para construir una base más sólida desde lo teórico.
— Investigar por mi cuenta juegos de gestión y estrategia para entender mejor cómo funcionan a nivel de diseño.
— Analizar juegos de terror y psicológicos desde un punto de vista más técnico, no solo como jugador.
— Seguir mejorando este proyecto, aplicando lo aprendido y corrigiendo cosas cuando haga falta, aunque implique rehacer partes.

Anexos

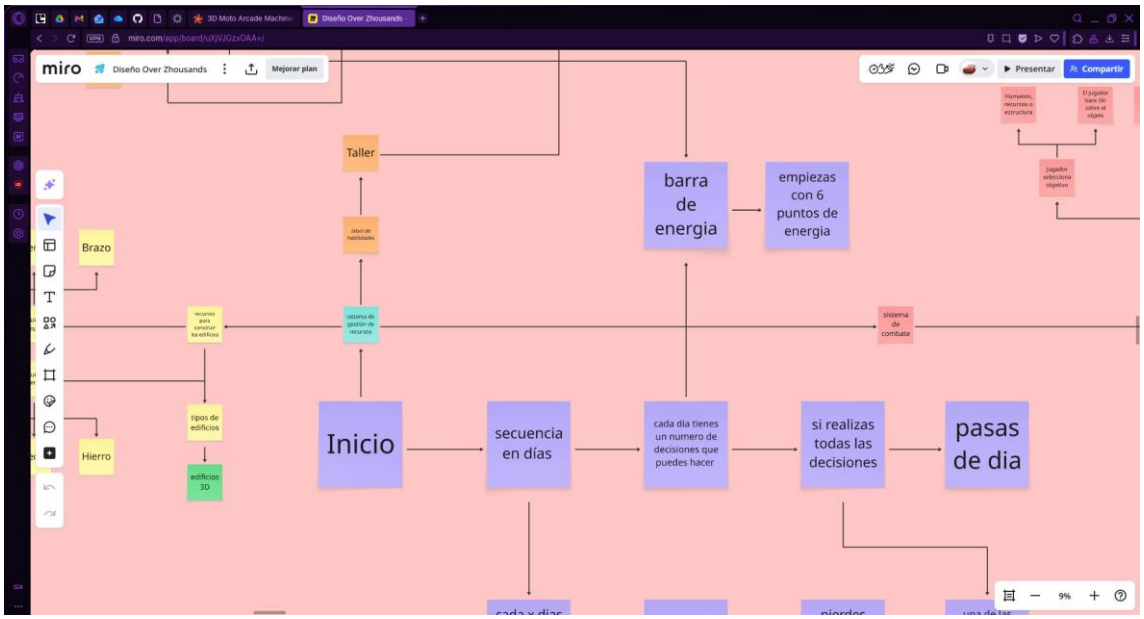

ZONA RECURSOS
ZONA JEFE
TORRETA
TORRETA
CAMPAMENTO
COMUNICACIÓN
TORRETA
TORRETA
PLAZA
TORRETA
CAMPAMENTO
BASE
CAMPAMENTO
TORRETA
CAMPAMENTO
80
80
miro

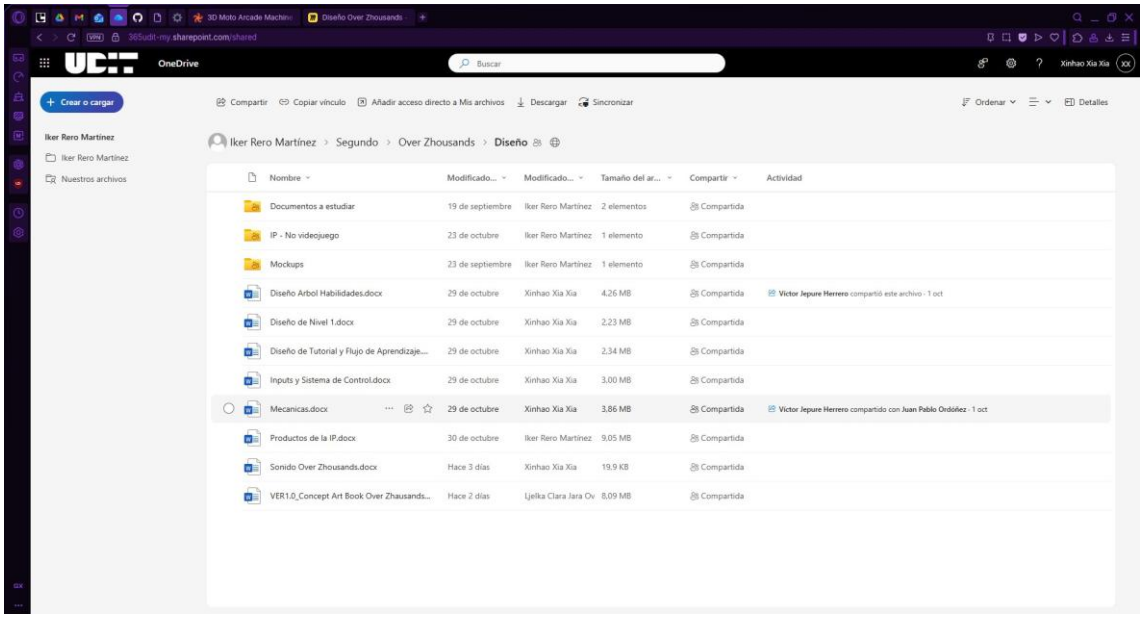

Degradado
Color negro
Zombie sentado triste
Introducir el nombre que quiere el jugador
miro

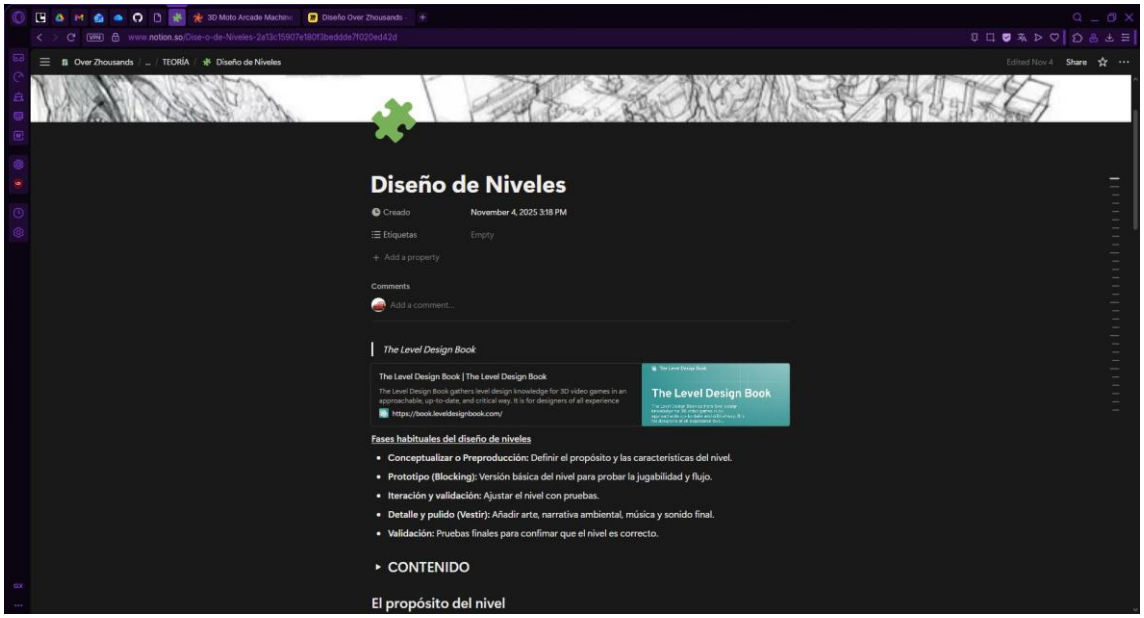

Color negro
Degradado
Zombie sentado triste
miro

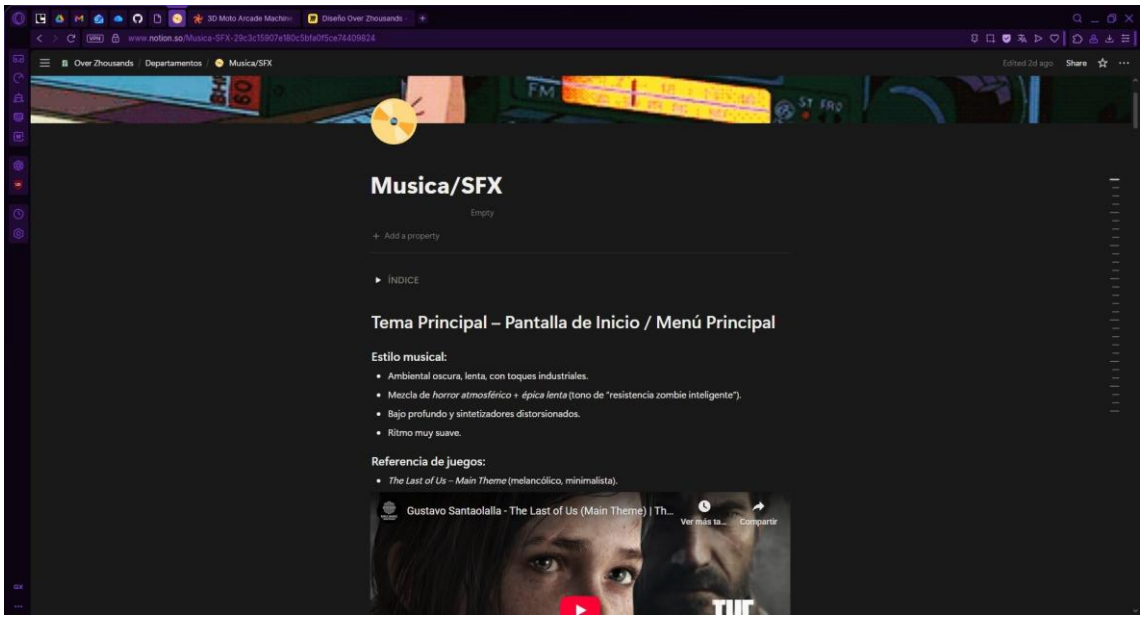

Bosque
Bosque
Montaña/Piedras
Aserradero
Cantera
Taller
BASE
Zombi
miro


Meta externa
Lo que el personaje hace
Meta interna
Lo que el personaje siente
Meta del jugador
Lo que el jugador experimenta
Zoe©
Controlar
Ejercito zombies
Ampliar la horda lo más posible
Expandir y mejorar el asentamiento
Evolucionar otros infectados
Personaje principal (zombie) del juego Over Zhousans (juego de taller)
Se siente solo
Porque es el único zombie con consciencia
Investiga evolución zombie
Devolver consciencia a su hermana
Venganza contra los humanos
Por subestimarla
Inteligente
Aprovechando que los humanos la subestima
Ansia
Consumir y convertir a los humanos lo antes posible
Frenesí
Progreso
Control total
Crecimiento
Enemistad
Influencia
Poder
miro


miro
Diseño Over Zhousands
Taller
barra de energia
empiezas con 6 puntos de energia
Brazo
Inicio
secuencia en días
cada dia tienes un numero de decisiones que puedes hacer
si realizas todas las decisiones
pasas de dia
Hierro
tipos de edificios
edificios 3D


OneDrive
Iker Rero Martínez › Segundo › Over Zhousands › Diseño
Documentos a estudiar
IP - No videojuego
Mockups
Diseño Arbol Habilidades.docx
Diseño de Nivel 1.docx
Diseño de Tutorial y Flujo de Aprendizaje....
Inputs y Sistema de Control.docx
Mecanicas.docx
Productos de la IP.docx
Sonido Over Zhousands.docx
VER1.0_Concept Art Book Over Zhausands...


Diseño de Niveles
Creado
November 4, 2025 3:18 PM
Etiquetas
Empty
Comments
The Level Design Book
The Level Design Book | The Level Design Book
Fases habituales del diseño de niveles
Conceptualizar o Preproducción: Definir el propósito y las características del nivel.
Prototipo (Blocking): Versión básica del nivel para probar la jugabilidad y flujo.
Iteración y validación: Ajustar el nivel con pruebas.
Detalle y pulido (Vestir): Añadir arte, narrativa ambiental, música y sonido final.
Validación: Pruebas finales para confirmar que el nivel es correcto.
CONTENIDO
El propósito del nivel


Musica/SFX
Empty
+ Add a property
▸ ÍNDICE
Tema Principal – Pantalla de Inicio / Menú Principal
Estilo musical:
Ambiental oscura, lenta, con toques industriales.
Mezcla de horror atmosférico + épica lenta (tono de "resistencia zombie inteligente").
Bajo profundo y sintetizadores distorsionados.
Ritmo muy suave.
Referencia de juegos:
The Last of Us – Main Theme (melancólico, minimalista).
Gustavo Santaolalla - The Last of Us (Main Theme) | Th...
Ver más ta...
Compartir


1
2
3
4
5
SICK
GECKO

UIButtonFeedBack.cs | UIAppearance.cs | LogoJuice.cs

Assembly-CSharp | UIButtonFeedback

```
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;

// Esto asegura que el script no cause errores de "Referencia nula"
[DisallowMultipleComponent]
Script de Unity | 0 referencias
public class UIButtonFeedback : MonoBehaviour, IPointerDownHandler, IPointerUpHandler, IPointerEnterHandler, IPointerExitHandler
{
    public float scaleFactor = 1.1f;
    public float speed = 15f;

    private Vector3 initialScale;
    private Vector3 targetScale;
    private bool isInitialized = false;

    Mensaje de Unity | 0 referencias
    void Start()
    {
        // Guardamos la escala real que le hayas puesto en el editor
        initialScale = transform.localScale;
        targetScale = initialScale;
        isInitialized = true;
    }

    // Usamos OnEnable por si desactivas y activas el botón
    Mensaje de Unity | 0 referencias
    void OnEnable()
    {
        if (isInitialized) transform.localScale = initialScale;
    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        // Movimiento suave
        transform.localScale = Vector3.Lerp(transform.localScale, targetScale, Time.deltaTime * speed);
    }

    0 referencias
    public void OnPointerDown(PointerEventData eventData) => targetScale = initialScale * scaleFactor;
    0 referencias
    public void OnPointerUp(PointerEventData eventData) => targetScale = initialScale;
    0 referencias
    public void OnPointerEnter(PointerEventData eventData) => targetScale = initialScale * (scaleFactor * 0.95f); // Un poco menos que el click
    0 referencias
    public void OnPointerExit(PointerEventData eventData) => targetScale = initialScale;
}
```

UIButtonFeedBack.cs | UIAppearance.cs | LogoJuice.cs

Assembly-CSharp | TerrariaLogoJuice

```
using UnityEngine;

Script de Unity (1 referencia de recurso) | 0 referencias
public class TerrariaLogoJuice : MonoBehaviour
{
    [Header("Ritmo de Respiración")]
    public float speed = 1.5f;
    public float scaleAmount = 0.08f; // Cuánto se infla (0.08 = 8%)

    [Header("Balanceo (Estilo Terraria)")]
    public float rotationAmount = 4f; // Grados de inclinación
    public float rotationSpeedMultiplier = 0.8f; // El balanceo suele ser más lento que el pulso

    private Vector3 initialScale;
    private float initialRotationZ;

    Mensaje de Unity | 0 referencias
    void Start()
    {
        initialScale = transform.localScale;
        initialRotationZ = transform.localEulerAngles.z;
    }

    Mensaje de Unity | 0 referencias
    void Update()
    {
        // Usamos Time.time con diferentes multiplicadores para romper la simetría perfecta
        float time = Time.time * speed;

        // --- PULSO (Escala) ---
        // Usamos el valor absoluto del seno para que parezca un latido constante
        float pulse = Mathf.Sin(time);
        float finalScale = 1f + (pulse * scaleAmount);
        transform.localScale = initialScale * finalScale;

        // --- BALANCEO (Rotación) ---
        // Al multiplicar el tiempo por otro valor (0.8f), el giro no coincide
        // exactamente con el pulso, creando ese efecto "vivo" de Terraria.
        float tilt = Mathf.Cos(time * rotationSpeedMultiplier) * rotationAmount;
        transform.localEulerAngles = new Vector3(0, 0, initialRotationZ + tilt);
    }
}
```